

Relatório Técnico para o Desenvolvimento da Plataforma de Jogos da Unidade Curricular de Fundamentos de Desenvolvimento de Software

André Kovtun 1250758, Diogo Moreira 1251752, Diogo Reis 1250895,
Gonçalo Matos 1241270

Grupo: 1
Turma: 1DA

Resumo: Este relatório descreve o desenvolvimento de uma plataforma de jogos em C++ baseada em programação orientada a objetos. A aplicação integra um sistema de autenticação, rankings globais e três jogos distintos: Blackjack, Jogo do Galo e Maior ou Menor. Todas as funcionalidades planeadas foram implementadas. O resultado final é uma aplicação estável e de fácil utilização que permite aos utilizadores jogar, monitorizar o seu desempenho e consultar classificações.

1 Introdução

Este projeto foi desenvolvido com o objetivo de aplicar os conhecimentos adquiridos ao longo do semestre, utilizando a linguagem de programação C++ e os princípios da Programação Orientada a Objetos. A plataforma consiste num sistema de jogos que permite aos utilizadores criar conta, iniciar sessão e aceder a três jogos distintos: Blackjack, Jogo do Galo e Maior ou Menor. Cada jogo possui regras e mecânicas próprias.

Para complementar a experiência, foi também implementado um sistema de rankings que regista e apresenta o desempenho dos jogadores de forma personalizada para cada jogo. O desenvolvimento da aplicação foi realizado de forma incremental, passando pelas fases de análise, design, implementação e testes. Ao longo destas etapas, foram produzidos diversos diagramas UML que auxiliaram na organização, estruturação e melhoria contínua da arquitetura do software.

Este relatório apresenta a solução desenvolvida, descrevendo a arquitetura do sistema, as principais funcionalidades implementadas, os testes realizados e os resultados obtidos durante o projeto.

2 Arquitetura do Sistema e Engenharia de Software

Nesta secção são apresentadas as principais decisões de arquitetura e organização adotadas durante o desenvolvimento da aplicação. São descritos os componentes que constituem o sistema, a forma como estes se relacionam entre si e a estrutura utilizada para garantir uma separação adequada de responsabilidades.

Para além disso, serão abordados alguns princípios que utilizamos durante o projeto, incluindo a organização do código, a utilização da programação orientada a objetos, a modularização dos diferentes componentes e os mecanismos implementados para o funcionamento desta plataforma. Por fim, são apresentados os diagramas e os modelos que representam a arquitetura e o funcionamento interno do sistema.

2.1 Arquitetura Geral do Sistema

A plataforma foi desenvolvida como uma aplicação de consola composta por vários módulos independentes, responsáveis pela gestão de utilizadores, execução dos jogos e manutenção dos rankings. A arquitetura foi concebida de forma modular para facilitar a manutenção, reutilização de código e futura expansão da aplicação.

Na figura 1 podemos ver a Arquitetura Geral da Plataforma de Jogos.

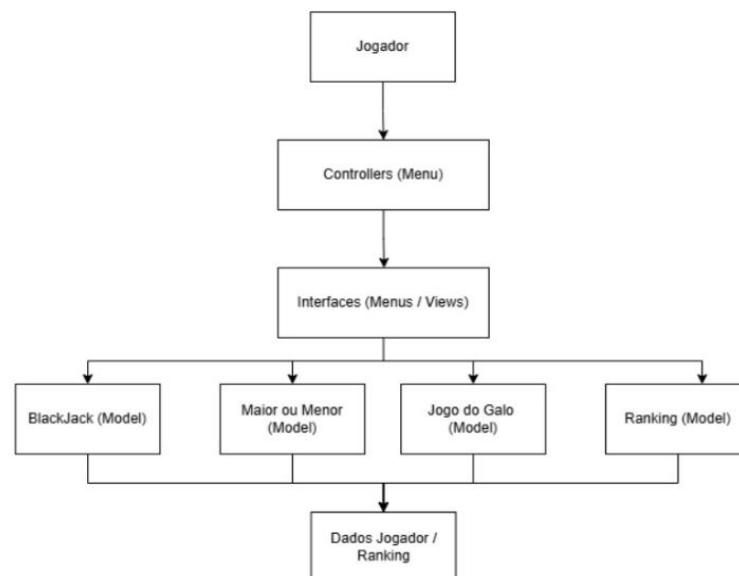


Figura 1 - Diagrama da Arquitetura Geral do Sistema

Como se pode observar no modelo estrutural, o fluxo de controlo da aplicação é dividida em quatro níveis lógicos de abstração:

- Ponto de Entrada (Jogador): O nível superior representa o ator que desencadeia a execução e interage diretamente com o processo do programa.
- Camada de Interface (Menus / Views): Funciona como um filtro isolador de Input/Output. É a única camada que comunica visualmente com o jogador.
- Camada Intermédia (Controllers): Atua como o motor do sistema. O controlador recebe as intenções da Interface que o jogador indica, valida o fluxo de navegação e delega o processamento pesado para as entidades do domínio.
- Camada de Domínio e Dados (Models): Composta pelas regras de cada jogo (BlackJack, Jogo do Galo, Maior ou Menor) e pelo módulo de Rankings. No nível base do diagrama, todas estas entidades convergem e dependem de um modelo comum de Dados do Jogador, assegurando o controlo centralizado de saldos e credenciais de inicialização.

2.2 O Padrão Arquitetural Model-View-Controller (MVC)

A plataforma foi desenhada/escrita em torno do padrão arquitetural **Model-View-Controller (MVC)**. Este padrão segmenta a aplicação em três camadas conceituais totalmente independentes, cada uma com um papel bem definido no ciclo de vida da execução:

- **Model:** É onde a lógica do jogo acontece. Guarda as informações e faz os cálculos matemáticos.
- **View:** Trata apenas do que aparece no ecrã. Usa o `std::cout` para mostrar os menus e o `std::cin` para ler os comandos. Não toma decisões lógicas.
- **Controller:** Faz a ponte entre o Model e a View. Controla o fluxo do programa, gere os turnos e decide qual é o próximo menu a abrir.

2.3 Princípios de Desenho de Software (GRASP)

Para organizar as responsabilidades das classes no código, foram aplicados três princípios do ecossistema GRASP:

Alta Coesão: Cada classe faz apenas uma tarefa bem definida.

- Exemplo: A lógica matemática do tabuleiro está isolada na classe Galo, enquanto o ecrã é tratado exclusivamente pela classe GaloView.

Baixo Acoplamento: Os módulos são independentes para evitar erros em cadeia.

- Exemplo: A View e o Model não se conhecem. Toda a comunicação é intermediada pelo Controller, permitindo alterar a interface sem estragar a lógica do jogo.

- **Princípio do Controller:** O Controller é o componente de software responsável por atuar como o intermediário direto entre a Interface do Utilizador (View) e o Model. No padrão de arquitetura MVC, o seu principal objetivo é garantir que a componente visual comunique diretamente com as regras de dados e estados do jogo.

2.4. Organização Estrutural do Código Fonte

A organização das pastas do projeto reflete diretamente a separação física do padrão MVC e a modularidade do sistema.

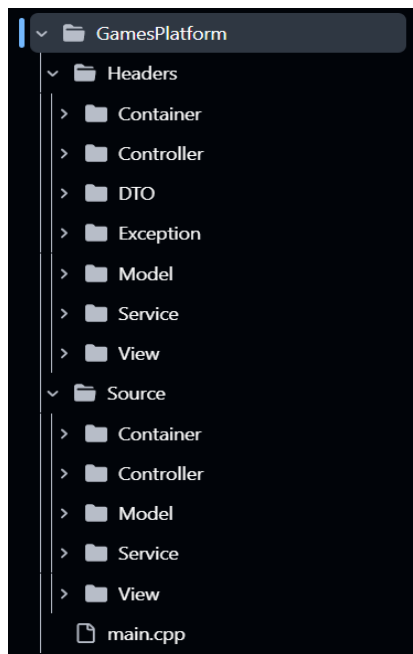


Figura 2 - Estrutura e organização dos diretórios do projeto

O código divide-se em duas pastas principais: **Headers** (ficheiros .h com as declarações) e **Source** (ficheiros .cpp com as implementações). Dentro delas, os ficheiros estão organizados por subpastas:

- **Model / View / Controller:** Separam o código das três camadas do MVC para não misturar a lógica com o ecrã.
- **Container:** Guarda as classes responsáveis por gerir conjuntos de dados em memória.
- **DTO (Data Transfer Objects):** Estruturas simples usadas apenas para passar dados de uma camada para a outra de forma segura.

- **main.cpp:** O ponto de entrada. Serve para criar o controlador principal e ligar o programa.

3. Diagrama de Casos de Uso (Perspetiva do Utilizador)

Este capítulo apresenta as ações que o Jogador pode realizar na nossa plataforma, explicadas através de um Diagrama de Casos de Uso em UML presente na figura 3.

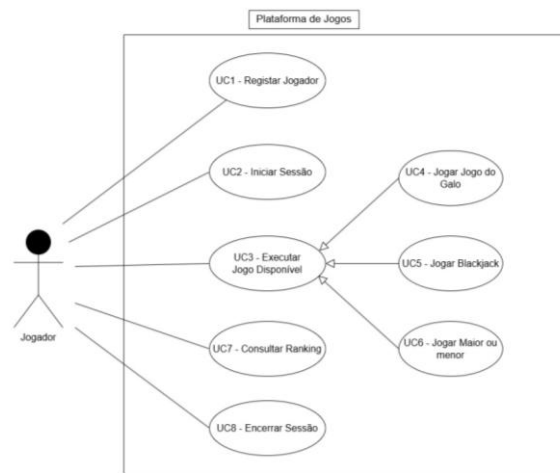


Figura 3 - Diagrama de Casos de Uso da plataforma na perspetiva do utilizador

O diagrama organiza-se da seguinte forma:

- **Ator (Jogador):** Representa qualquer utilizador que interaja com o terminal da plataforma.
- **Fronteira do Sistema (Plataforma de Jogos):** Representa o limite da nossa aplicação de consola, onde correm todos os módulos lógicos.
- **Ações Principais:** O utilizador pode Criar Conta (UC1), Iniciar Sessão (UC2), Consultar Ranking (UC7) e Encerrar Sessão (UC8).
- **Relação de Especialização (Executar Jogo):** O caso de uso geral "Executar Jogo Disponível (UC3)" divide-se em três ações específicas:
 - Jogar Jogo do Galo
 - Jogar Blackjack
 - Jogar Maior ou Menor

Esta abordagem de herança no diagrama mostra que a plataforma foi desenhada de forma modular: o sistema sabe como iniciar um jogo, e cada jogo específico apenas implementa as suas próprias regras e funções

4 Conceção e Implementação dos Módulos

Neste capítulo é detalhado o funcionamento interno, as regras e a lógica de programação de cada módulo da plataforma.

4.1 Módulo da Gestão de Utilizadores

Para gerir os perfis dos utilizadores e controlar os acessos, foi desenvolvida a classe JogadorContainer. Este componente centraliza as operações de registo e login em memória através de um vetor dinâmico (`std::vector<Jogador>`).

O funcionamento lógico do container baseia-se em três pontos:

- **Registo (add):** Recebe o username e a password. Antes de criar o jogador, invoca o método `usernameExiste` para garantir que o nome não está repetido. Se já existir, rejeita o registo (retorna false).
- **Autenticação (validarCredenciais):** Percorre a lista de jogadores e compara as credenciais introduzidas. Se encontrar uma correspondência exata, devolve o ponteiro do Jogador para iniciar a sessão ativa. Se falhar, devolve (nullptr).
- **Validação (usernameExiste):** Método auxiliar que faz uma pesquisa linear no vetor para verificar se um determinado *username* já foi registado no sistema.

4.2 Módulo do Jogo do Galo

Este módulo é responsável por executar o Jogo do Galo tradicional, controlando o tabuleiro e a alternância de jogadas. O seu funcionamento está dividido de forma a usar o padrão MVC:

- **Model (Galo):** Guarda o estado do jogo numa matriz estática de 3x3 posições. Contém a lógica pura para:
 - Verificar se a posição escolhida está livre.
 - Validar se houve um vencedor.
 - Detetar situações de empate.
- **View (GaloView):** Trata apenas da parte visual no ecrã. Limita-se a desenhar as linhas do tabuleiro na consola e a pedir as coordenadas (linha e coluna) ao jogador.
- **Controller (GaloController):** Controla o ciclo do jogo. É ele que faz a alternância da vez entre os jogadores, recebe as coordenadas da View, manda o Model validar e atualizar a matriz, e termina a partida quando há um vencedor ou então um empate.

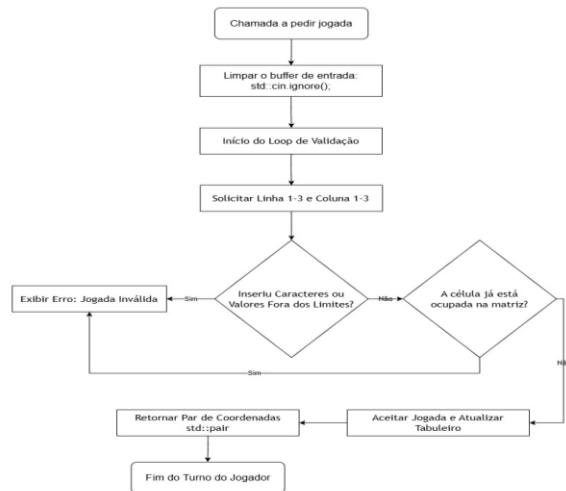


Figura 4 - Fluxograma do Jogo do Galo

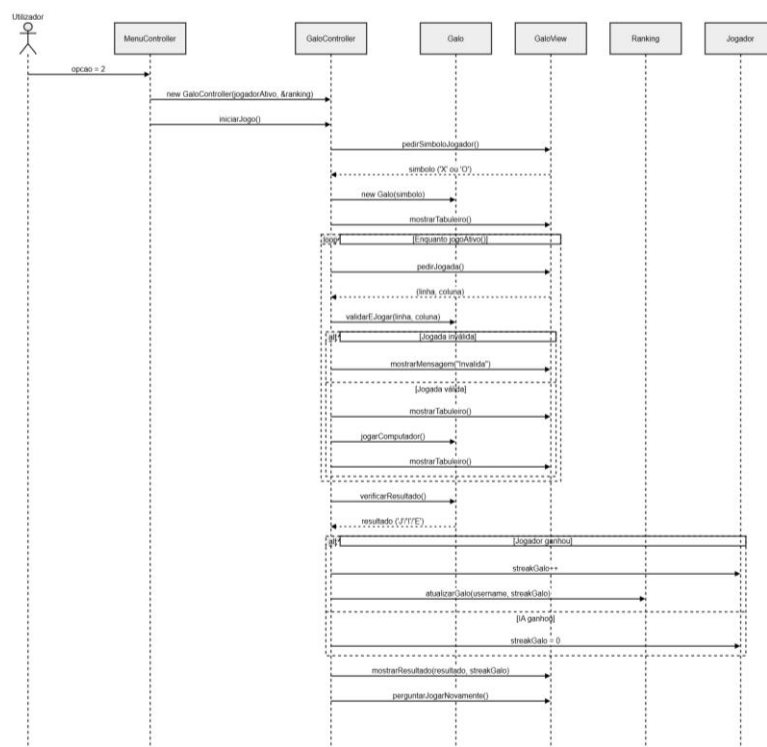


Figura 5 - Diagrama de Sequência do Jogo do Galo

4.3 Módulo do Blackjack

Este módulo é responsável por gerir o jogo de cartas Blackjack, controlando o baralho, os saldos e as jogadas do dealer. O funcionamento está dividido através do padrão MVC:

- **Model (Blackjack):** Guarda o estado do baralho e as cartas na mão do jogador e do Dealer. Contém a lógica pura para:
 - Calcular o valor total da mão.
 - Validar se o jogador/dealer ultrapassou os 21 pontos (bust).
 - Controlar o saldo e o valor da aposta atual.
- **View (BlackjackView):** Trata estritamente do output no terminal. Mostra graficamente em texto as cartas que estão na mesa, as mãos atuais de cada um e lê a decisão do jogador (hit ou stand).
- **Controller (BlackjackController):** Coordena o fluxo da partida. Distribui as cartas iniciais, gerencia a vez do jogador (hit para pedir carta ou stand para manter) e executa a lógica automática do Dealer (que pede cartas obrigatoriamente até atingir um mínimo de 17 pontos). No final, decide o vencedor e atualiza o saldo no Model.



Figura 5 – Fluxograma de jogo e decisões do Blackjack

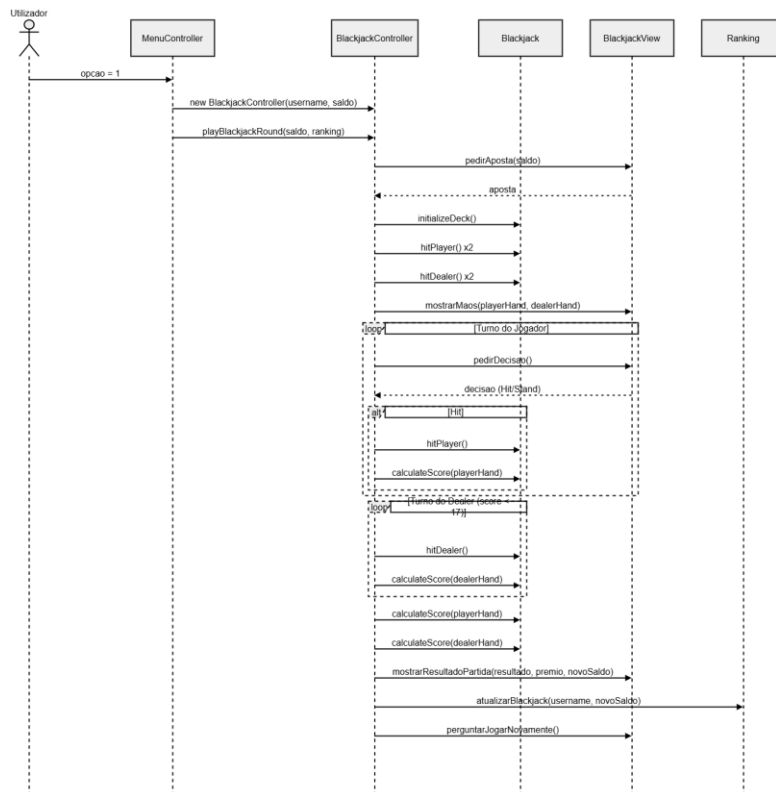


Figura 6 - Diagrama de Sequência do Blackjack

4.4 Módulo do Maior ou Menor

Este módulo controla o jogo do Maior Ou Menor, onde o utilizador tenta adivinhar um número secreto gerado pelo sistema num intervalo de números (1 a 100). A estrutura segue o padrão MVC:

- **Model (MaiorOuMenor):** Contém as regras e os dados do jogo. É responsável por:
 - Gerar o número secreto de forma aleatória no início da partida.
 - Validar o palpite do jogador, comparando-o com o número secreto.
 - Contabilizar o número de tentativas guardadas.
- **View (MaiorOuMenorView):** Trata apenas da interação com o ecrã. Pede ao jogador para digitar um número e exibe as dicas textuais fornecidas pelo sistema (ex: "O número secreto é maior" ou "O número secreto é menor").
- **Controller (MaiorOuMenorController):** Controla o ciclo principal do jogo. Solicita o input do jogador através da View, envia o palpite para o Model validar e, com base na resposta, decide se o jogo continua (pedindo um novo palpite com dicas) ou se termina.

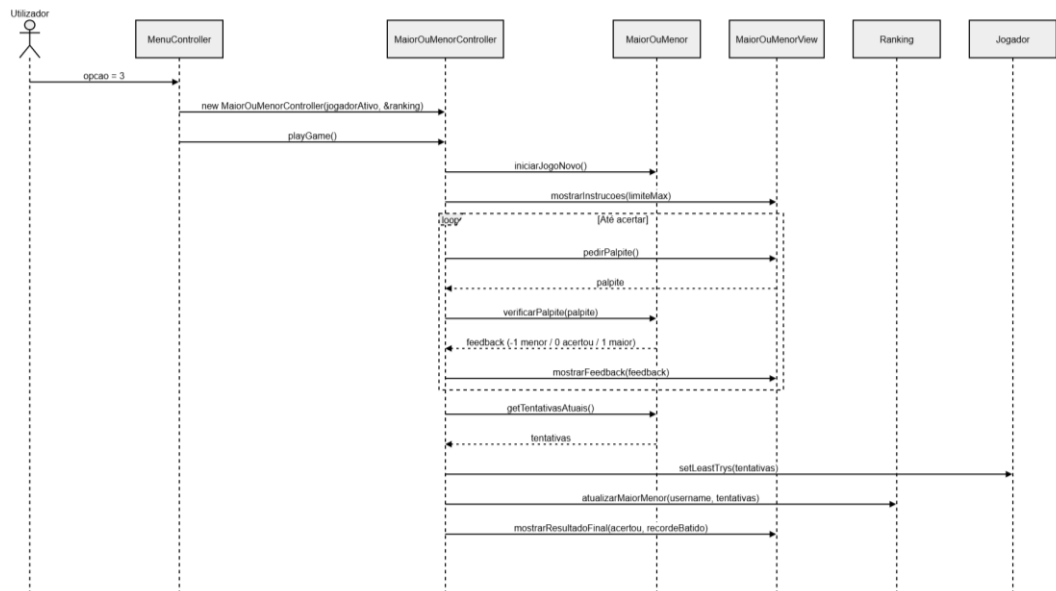


Figura 7 - Diagrama de Sequência do Maior ou Menor

4.5 Módulo de Rankings

Este módulo é responsável por organizar e mostrar a classificação geral dos utilizadores com base nos resultados obtidos nos jogos. A estrutura segue o padrão MVC:

- **Model:** Não precisa de uma classe de dados nova. Em vez disso, consome diretamente os dados guardados no JogadorContainer, acedendo à lista de utilizadores através do método getJogadores().
- **View:** Trata apenas da formatação visual no ecrã. Desenha uma tabela organizada no terminal para mostrar o Top de jogadores, exibindo a posição, o username e os respetivos pontos/vitórias.
- **Controller:** Faz o trabalho de processamento. Pede a lista de jogadores ao Container, ordena-os de forma decrescente (do que tem mais pontos para o que tem menos) usando funções de ordenação do C++, e envia os dados já ordenados para a View desenhar.

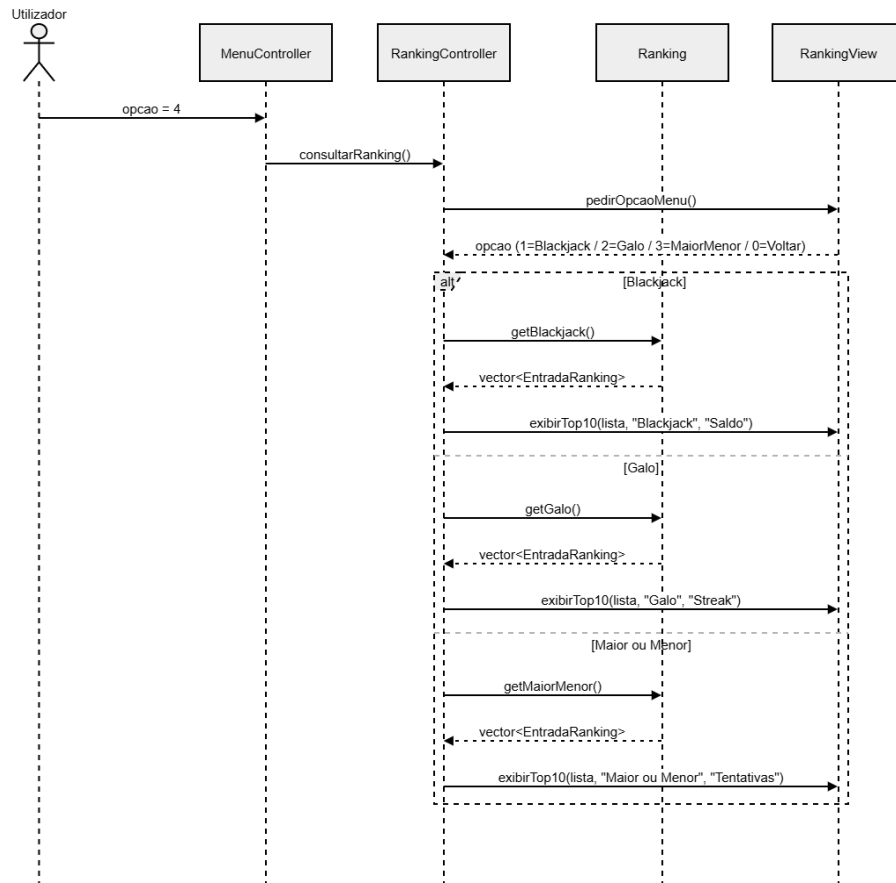


Figura 8 – Diagrama de Sequência do Ranking

5 Estruturas de Dados e Otimização Algorítmica

Este capítulo demonstra a aplicação prática de algoritmos fundamentais e a gestão eficiente do tempo de vida das variáveis em memória.

5.1 Algoritmo de Ordenação (Bubble Sort)

Para a gestão e sustentabilidade dos rankings, foi implementado o algoritmo de ordenação Bubble Sort no Modelo do ranking. A ordenação é executada logo após a inserção ou atualização de um registo, garantindo que os vetores em memória se mantêm constantemente estruturados.

Como o critério de vitória difere consoante o jogo, a polaridade da ordenação foi invertida inteligentemente através dos operadores relacionais:

```
75 void Ranking::atualizarMaiorMenor(std::string username, int tentativas) {
76     bool encontrado = false;
77
78     for (int i = 0; i < (int)rankingMaiorMenor.size(); i++) {
79         if (rankingMaiorMenor[i].username == username) {
80             encontrado = true;
81             if (tentativas < rankingMaiorMenor[i].valor) {
82                 rankingMaiorMenor[i].valor = tentativas;
83             }
84             break;
85         }
86     }
87
88     if (!encontrado) {
89         EntradaRanking nova;
90         nova.username = username;
91         nova.valor = tentativas;
92         rankingMaiorMenor.push_back(nova);
93     }
94
95     for (int i = 0; i < (int)rankingMaiorMenor.size() - 1; i++) {
96         for (int j = 0; j < (int)rankingMaiorMenor.size() - 1 - i; j++) {
97             if (rankingMaiorMenor[j].valor > rankingMaiorMenor[j+1].valor) {
98                 EntradaRanking temp = rankingMaiorMenor[j];
99                 rankingMaiorMenor[j] = rankingMaiorMenor[j+1];
100                rankingMaiorMenor[j+1] = temp;
101            }
102        }
103    }
104
105    if ((int)rankingMaiorMenor.size() > TOP_MAX) {
106        rankingMaiorMenor.pop_back();
107    }
108 }
```

Figura 9 – Exemplo da ordenação do ranking (Maior Ou Menor)

Ordem Crescente: Aplicada de forma cirúrgica no **Maior ou Menor**, invertendo a lógica para o operador de maiorização (>). Dado que o topo do ranking deve pertencer a quem gastou menos tentativas para adivinhar o número, os valores mais altos são empurrados para o fim.

Ordem Decrescente (Maior para o Menor): Aplicada no Blackjack (Maior Saldo) e Jogo do Galo (Maior Streak). Utiliza a verificação de menorização (<) para garantir que os jogadores com pontuações mais altas subam para o topo do vetor.

Para evitar o crescimento infinito dos vetores em memória e otimizar o espaço ocupado, foi definida uma constante (TOP_MAX = 10). Logo após o algoritmo ordenar a coleção, o método pop_back() remove o elemento que ficou em 11º lugar, truncando a estrutura estritamente para o Top 10.

6 Tratamento de Erros e Resolução de Desafios Técnicos

Esta secção aborda os desafios técnicos encontrados durante o desenvolvimento da plataforma e as soluções aplicadas para garantir a estabilidade e a robustez do sistema contra falhas durante a sua execução.

6.1 Desafio do Buffer de I/O

Um dos problemas mais recorrentes no desenvolvimento da nossa plataforma ocorreu quando o jogador inseria um tipo de dados inválido — por exemplo, introduzir caracteres alfabéticos (letras) quando o sistema aguarda um input numérico (como o valor de uma aposta no Blackjack ou a escolha de uma opção de menu).

Quando isso acontece, o fluxo de entrada do `std::cin` entra num estado de falha (ativando o *fail bit*) e o caractere inválido fica retido no buffer. Se não for tratado, o programa ignora os pedidos de input seguintes e entra num ciclo infinito de leitura de lixo.

Para garantir o saneamento do fluxo de entrada, foi implementada uma rotina de validação e limpeza:

```
78     int opcao;  
79     while (!(std::cin >> opcao) || (opcao != 0 && opcao != 1)) {  
80         std::cin.clear();  
81         std::cin.ignore(n:10000, delim: '\n');  
82         std::cout << "Opcao invalida (0 ou 1): ";  
83     }  
84     std::cin.ignore(n:10000, delim: '\n');  
85     return opcao == 1;  
86 }
```

Figura 10 – Código para resolver problema de Input

`std::cin.clear()` redefine o estado do fluxo, permitindo que o programa volte a aceitar inputs.

`std::cin.ignore(10000, '\n')` limpa efetivamente o lixo armazenado no buffer até encontrar uma quebra de linha. Esta combinação isola o erro de digitação do utilizador e impede a interrupção abrupta da execução.

6 Testes e Validação

Para garantir que a plataforma funciona corretamente e não vai abaixo com erros do utilizador, a equipa realizou testes práticos divididos em três áreas: Validação de Regras, Robustez de Input e Integração do Sistema.

Módulo	Cenário Teste	Comportamento Esperado	Resultado
Autenticação	Criar conta com um username que já existe.	O sistema deve rejeitar e pedir um username novo.	Sucesso
Autenticação	Fazer Login com a password errada.	O sistema nega o acesso e devolve nullptr no apontador do jogador	Sucesso
Jogo do Galo	Tentar jogar numa coordenada inválida (ex: linha 4) ou ocupada.	O método validarEJogar() deve retornar false e impedir a jogada.	Sucesso
Jogo do Galo	Tabuleiro preenchido sem nenhum vencedor de 3 em linha.	O jogo deve detetar o empate ('E') através do verificarResultado().	Sucesso
Blackjack	Jogador pede cartas (<i>hit</i>) e ultrapassa os 21 pontos.	O modelo calcula o <i>bust</i> , encerra o turno e retira o valor apostado do saldo.	Sucesso
Maior ou Menor	Introduzir um palpite igual ao número secreto gerado.	O jogo valida o acerto, mostra o número de tentativas e atualiza o ranking.	Sucesso
Geral (I/O)	Digitar letras ou símbolos num campo que pede números.	O loop com <code>std::cin.clear()</code> e <code>ignore()</code> deve limpar o erro sem crashar.	Sucesso

Tabela 1 - Casos de teste da plataforma de jogos

6.1 Análise Crítica dos Testes

Os testes mostraram que a separação de código pelo padrão MVC facilitou muito a detecção de erros. Como a lógica (*Model*) não depende do ecrã (*View*), conseguimos testar as regras de vitória dos jogos isoladamente, sem precisar de simular a interface completa da consola.

O programa demonstrou ser estável e resiliente, lidando bem com os erros de digitação mais comuns e mantendo a integridade dos dados dos jogadores (saldos e pontuações) entre a troca de menus e de jogos.

7 Conclusões e Trabalho Futuro

O desenvolvimento desta plataforma de jogos em C++ foi fundamental na consolidação de competências práticas de Programação Orientada a Objetos (POO) e Otimização Algorítmica, também como trabalho em grupo. O projeto foi positivo, tendo atingido com sucesso todos os requisitos funcionais exigidos na especificação técnica.

O fator crítico para um bom funcionamento e estabilidade da aplicação foi a implementação da arquitetura MVC. Inicialmente, a estrutura do projeto não era exatamente como está agora a final, porém, depois de algum debate entre o grupo, chegámos à conclusão que seria mais fácil haver separação de pastas (Headers e Source) e, da mesma forma, a separação das pastas (Model, Controller, View, etc.). Com isso, melhorámos bastante o trabalho e a organização, tal como a compreensão do código por parte de todos os membros da equipa.

Esta organização modular permitiu que o desenvolvimento avançasse sem conflitos. Conseguimos programar e encaixar os três jogos (Blackjack, Jogo do Galo e Maior ou Menor) de forma independente. Quando era preciso corrigir um bug ou alterar o ecrã de um jogo, sabíamos exatamente a que pasta recorrer, sem o risco de estragar os jogos que já estavam feitos ou o sistema global de Rankings.

Por fim, resolvemos os problemas finais para ter um código 100% funcional sem erros aparentes, que oferece uma boa experiência ao jogador.